SIMATS ENGINEERING
ASSESSMENT TOOL 1
COURSE NAME: Artificial Intelligence COURSE CODE: CSA1741

CO1-AT1-Analytical Problem Solving

Register Number	**192412370**
Name	**Madhan R**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks: 50**

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

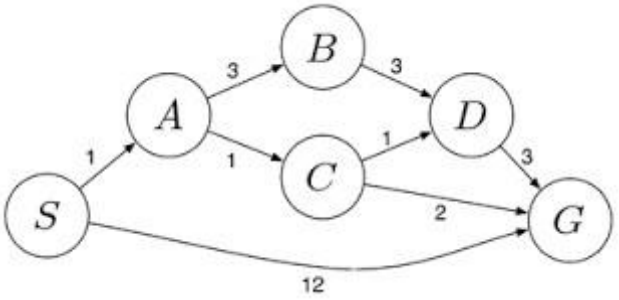
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

   The delivery network is represented as a weighted graph:



1. Water Jug Problem (4-Gallon and 3-Gallon Jugs)

Introduction

The Water Jug Problem is a classical AI problem-solving example. The objective is to obtain exactly 2 gallons of water in the 4-gallon jug using a 4-gallon jug and a 3-gallon jug without any measuring marks.

State representation

Represent each state as:

\[(x,y) \]

where:

- \(x\) = water in the 4-gallon jug.
- \(y\) = water in the 3-gallon jug.

Initial state: \((0,0)\)

Goal state: Any state where \(x=2\).

Allowed operations

1. Fill either jug completely.
2. Empty either jug.
3. Pour water from one jug into the other until the source is empty or the destination is full.

Step-by-step solution

Step	Operation	4-gallon jug (x)	3-gallon jug (y)
1	Start with both jugs empty	0	0
2	Fill the 3-gallon jug	0	3
3	Pour the 3-gallon jug into the 4-gallon jug	3	0
4	Fill the 3-gallon jug again	3	3
5	Pour from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full	4	2
6	Empty the 4-gallon jug	0	2
7	Pour the remaining 2 gallons into the 4-gallon jug	2	0

Final answer

The 4-gallon jug contains exactly 2 gallons, so the goal is achieved.

Conclusion: The Water Jug Problem can be solved using state-space search by applying valid operations until the goal state is reached.

2. Mars Rover – Intelligent Agent Analysis

Introduction

A Mars Rover is an autonomous robot designed to explore the surface of Mars, collect scientific data, and analyze soil and rock samples. It uses sensors, cameras, and scientific instruments to make decisions in an environment where communication with Earth is delayed.

(i) What are the percepts of this agent?

Percepts are the information received by an intelligent agent through its sensors.

The Mars Rover perceives:

- Images of the Martian surface through cameras.
- Distance and obstacles using navigation sensors.
- Soil and rock composition through scientific instruments.
- Temperature, pressure, and environmental conditions.
- Battery level and equipment status.
- Wheel movement and rover position.
- Signals and instructions received from Earth.

(ii) Characterize the operating environment

Environment property	Classification	Reason
Observability	Partially observable	The rover cannot observe the entire planet.
Determinism	Partly stochastic	Terrain and equipment outcomes may be uncertain.
Episodic/sequential	Sequential	Each action affects future decisions.
Static/dynamic	Dynamic	Environmental conditions can change.

Discrete/continuous	Hybrid	Actions may be discrete, while movement and sensor readings are continuous.
Single/multi-agent	Primarily single-agent	The rover operates autonomously, although Earth-based teams provide support.
Known/unknown	Partially unknown	Terrain and scientific properties may be undiscovered.

(iii) What actions can the agent take?

The Mars Rover can perform the following actions:

1. Move forward, backward, or turn.
2. Capture images and scan the surrounding terrain.
3. Avoid obstacles and select a safe route.
4. Collect soil and rock samples.
5. Analyze samples using onboard scientific instruments.
6. Transmit images, scientific results, and status reports to Earth.
7. Manage battery power and enter a safe mode when necessary.

(iv) How can the performance of the agent be evaluated?

The performance can be evaluated using:

- Number of scientific samples collected and analyzed.
- Accuracy and usefulness of scientific measurements.
- Distance explored and area covered.
- Safety of navigation and number of collisions avoided.
- Energy efficiency and battery life.
- Amount of useful data transmitted to Earth.
- Mission objectives completed within the available time and resources.

(v) Most suitable agent architecture

A model-based learning agent with goal-based and utility-based decision-making is most suitable.

- The model-based component maintains an internal map and remembers previous observations.
- The goal-based component plans to reach scientific targets and collect samples.
- The utility-based component balances scientific value, safety, time, and energy consumption.
- The learning component improves decisions using experience from earlier exploration.

Conclusion: A hybrid intelligent agent allows the Mars Rover to explore independently, make safe decisions, conserve energy, and collect valuable scientific information despite limited communication with Earth.

3. Eight Queens Problem – Formulation and Search Strategies

Introduction

The Eight Queens Problem is a classical constraint satisfaction problem in Artificial Intelligence. The objective is to place eight queens on an 8×8 chessboard so that no two queens attack one another.

A queen can attack another queen in the same row, column, or diagonal.

Problem formulation

1. Initial state: An empty chessboard.

2. State representation: Place one queen in each column. Represent the state as:

$$ Q=[q_1,q_2,\ldots,q_8] $$

Here, q_i is the row number of the queen placed in column i.

3. Initial state: $Q=[]$, with no queens placed.

4. Actions: Place a queen in a safe row of the next column.

5. Transition model: Add a queen to the selected row and column if it does not violate the constraints.

6. Goal test: All eight queens are placed without conflicts.

7. Path cost: Usually, each queen placement costs 1. The objective is to find a valid configuration rather than minimize a varying path cost.

Constraints

For any two queens in columns i and j:

Row constraint:

$$ q_i \ne q_j $$

Diagonal constraint:

$$ |q_i-q_j|\ne |i-j| $$

Because only one queen is placed per column, column conflicts are automatically avoided.

Search strategy: Backtracking Search

Backtracking is effective because it places one queen at a time and immediately rejects unsafe placements.

Steps:

1. Start with an empty board.
2. Place a queen in the first column.
3. Move to the next column and choose a safe row.
4. Check row and diagonal constraints.
5. If no safe row exists, remove the previous queen and try another row.
6. Repeat until all eight queens are placed.

Example of a valid solution

One valid configuration is:

\[Q=[1,5,8,6,3,7,2,4] \]

Each number represents the row of the queen in its corresponding column.

Illustration of a valid eight-queens placement. Each column contains one queen, and no two queens share a row or diagonal.

Other search strategies

- Breadth-First Search (BFS): Explores partial configurations level by level, but can consume considerable memory.
- Depth-First Search (DFS): Explores one placement sequence deeply before backtracking.
- Backtracking Search: Checks constraints at each step and prunes invalid configurations early.
- Local Search: Min-conflicts can start with eight queens and repeatedly move a conflicting queen to reduce conflicts.

Conclusion

The Eight Queens Problem can be solved using constraint-based backtracking. By placing one queen per column and checking row and diagonal constraints, invalid configurations are eliminated early, reducing unnecessary search.

4. OLA Cab Booking – Problem-Solving Agent and Pseudocode

Introduction

An OLA cab booking application helps a customer travel from a source location to a destination. The customer can select different cab types, such as Mini, Micro, Sedan, Shared, or Prime, based on fare, comfort, availability, and travel time.

The most suitable agent is a utility-based problem-solving agent, because it can compare different cab options and select the one that best matches the customer's preferences.

Problem formulation

- Initial state: Customer enters the pickup and destination locations.

- Goal state: Customer reaches the destination using a suitable cab.
- Actions: Select a cab, compare fares, check availability, and book the cab.
- Constraints: Budget, comfort, travel time, availability, and customer preference.
- Performance measure: Suitable cab selection, reasonable fare, low waiting time, and customer satisfaction.

Pseudocode

```
BEGIN

INPUT pickup_location
INPUT destination
INPUT customer_preferences

GET available_cabs()

FOR each cab IN available_cabs:
    GET fare(cab)
    GET estimated_time(cab)
    GET comfort_level(cab)
    GET availability(cab)

    IF cab is available:
        CALCULATE utility(cab) using:
            fare
            travel_time
            comfort
            customer_preferences

SELECT cab with highest utility

DISPLAY selected cab, fare, and estimated time

IF customer confirms booking:
    BOOK selected cab
    DISPLAY booking confirmation
ELSE:
    DISPLAY available cab options

END
```

Explanation

1. The customer enters the pickup and destination.
2. The application retrieves available cab types.
3. Each cab is evaluated based on price, comfort, travel time, and availability.
4. The utility-based agent ranks the suitable options.
5. The customer selects and confirms a cab.
6. The application completes the booking and provides confirmation.

Conclusion

A utility-based problem-solving agent is suitable because it compares multiple alternatives and selects a cab according to the customer's preferences. A goal-based component can also ensure that the destination is reached.

5. Logistics Delivery Network – Uniform Cost Search (UCS)

Introduction

Uniform Cost Search is an uninformed search algorithm used to find the least-cost path between a starting node and a goal node in a weighted graph. In a logistics network, warehouses are represented as nodes, and transport routes are represented as edges with associated costs.

Given:

- Starting warehouse: S
- Destination warehouse: G
- Route cost: Fuel cost + travel-time cost
- Objective: Find the minimum-total-cost path from S to G.

UCS algorithm

UCS uses a priority queue ordered by the total accumulated path cost $g(n)$.

$$ g(n)=\sum \text{costs of edges along the path} $$

Pseudocode

```
UCS(Graph, S, G):

    Create an empty priority queue PQ
    Insert (S, 0) into PQ
    Set cost[S] = 0
    Set parent[S] = NULL

    WHILE PQ is not empty:

        Remove node n with minimum cost from PQ

        IF n = G:
            RETURN reconstructed path and cost[G]

        FOR each neighbour v of n:

            new_cost = cost[n] + edge_cost(n, v)

            IF v is unvisited OR new_cost < cost[v]:
                cost[v] = new_cost
                parent[v] = n
                Insert or update (v, new_cost) in PQ

    RETURN "No path exists"
```

Step-by-step procedure

1. Initialize the priority queue with S at cost 0.
2. Remove the node with the lowest accumulated cost.
3. Examine all neighbouring warehouses.
4. Calculate the new cost for each route.
5. Update the priority queue if a cheaper path is found.
6. Continue until G is removed from the priority queue.

7. Trace the parent nodes to reconstruct the least-cost path.

Important note about the given graph

The graph image and its edge weights are not visible in your message. Therefore, the exact expansion order, optimal path, and minimum total cost cannot be calculated reliably yet.

Please upload the graph image for Question 5, and I can solve it step by step with the priority queue updates and final minimum-cost route.

Conclusion

UCS is suitable for the logistics problem because it considers the total accumulated route cost rather than simply choosing the nearest warehouse. With non-negative edge costs, UCS finds the optimal least-cost path when the goal is removed from the priority queue.

Code of Conduct:

I Madhan R\192412370 (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Madhan R

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem	10
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method	10

Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process	12.5
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations	10
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation	7.5